Compiler Construction Lab

Project Report

Design and Implementation of a Mini Programming Language Compiler using Flex and Bison

Team: ParseMasters

Metropolitan University, Bangladesh
Department of Computer Science and Engineering

Name	Student ID	GitHub
Mashuda Siddika Maisha	231-115-119	mashudasiddikamaisha
Nusrath Jahan Nowshin	231-115-088	Nowshin03
Najmun Nahar	231-115-112	najmuntushi-hue

Table of Contents

1. Introduction

Compiler construction is an important area of Computer Science that explains how programs written in a human-readable programming language can be analyzed and transformed into a lower-level representation that can be processed by a computer. A compiler is generally divided into several phases, including lexical analysis, syntax analysis, semantic analysis, intermediate code generation, optimization, and code generation.

The Compiler Construction Lab project at Metropolitan University requires students to integrate the major front-end phases of compiler construction into one working system. The project specifically requires a compiler for a fixed custom programming language and does not require machine code, assembly code, register allocation, linking, or executable generation. The main purpose is to demonstrate how the different compiler phases communicate with each other.
Our project ParseMasters is a mini compiler front-end developed using the C programming language, Flex, Bison, GCC, and GNU Make. The compiler accepts source programs written in the specified custom language and processes them through lexical analysis, syntax analysis, Abstract Syntax Tree construction, symbol-table processing, semantic analysis, and Three Address Code generation.

The project implements the six major required modules:
Lexical Analyzer
Syntax Analyzer
Abstract Syntax Tree
Symbol Table
Semantic Analyzer
Three Address Code Generator

The project supports integer, floating-point, and Boolean data types. It also supports variable declarations, assignments, arithmetic expressions, relational expressions, logical expressions, conditional statements, while loops, print statements, and nested blocks.
The project repository is organized according to compiler phases. The src directory contains separate modules for the lexer, parser, AST, semantic analyzer, symbol table, and TAC generator. The repository also contains examples, tests, documentation, and build-related files.

The project was developed as a team effort. Each member was responsible for specific modules while collaborating through GitHub. The division of work allowed the team to develop the compiler incrementally and integrate the separate modules into a single compiler system.

2. Objectives

The main objective of ParseMaster is to demonstrate the practical implementation of the fundamental phases of a compiler front-end using Flex and Bison.

The specific objectives are:

- ## Lexical Analysis

To develop a lexical analyzer using Flex that recognizes keywords, identifiers, constants, operators, delimiters, comments, and whitespace.

- ### Syntax Analysis

To design and implement a Context-Free Grammar for the custom language using Bison and detect syntactic errors with line numbers.

- ### Abstract Syntax Tree Construction

To construct an Abstract Syntax Tree from successfully parsed source code and provide a readable textual representation of the tree.

- ### Symbol Table Management

To maintain information about declared variables including their names, types, declaration lines, and scopes.

- ### Semantic Analysis

To detect semantic errors that cannot be identified by the grammar alone. These include:

Undeclared variable use
Variable redeclaration
Scope violations
Type mismatches
Invalid assignments
Invalid expressions
Invalid Boolean conditions

- ### Intermediate Code Generation

To generate Three Address Code from a semantically valid AST using temporary variables and labels.

- ### Error Reporting

To provide clear and understandable error messages so that programmers can identify lexical, syntax, and semantic problems.

- ### Compiler Integration

To integrate all required modules into a single buildable compiler using a Makefile and GNU development tools.

3. Language Specification

As mentioned in the introduction, the programming language used in ParseMaster is defined by the Compiler Construction Lab Project Manual and was not designed by our team. Our task was to carefully follow the given language specification, create a clear grammar based on it, and implement all the required language features without adding or removing anything from the original specification.

- ### Data Types

Type	Description	Examples
int	Signed integer	10, 25
float	Floating-point number	3.14, 5.65
bool	Boolean	true, false

Supported Statements

The language supports

- Variable declarations
- Variable assignments
- Arithmetic expressions
- Relational expressions
- Logical expressions
- if statements
- if-else statements
- while loops
- print statements
- Nested blocks

- Examples:

```
int x;
float y;
bool flag;

x = 10;
y = 3.14;
flag = true;

print x;
```

Operator

Arithmetic Operators

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus

Relational Operators

Operator	Meaning
<	**Less than**
>	**Greater than**
<=	**Less than or equal**
>=	**Greater than or equal**
==	**Equal**
!=	**Not equal**

Logical Operators

Operator	Meaning
&&	Logical AND
||	Logical OR
!	Logical NOT

- ## Identifiers

An identifier must begin with a letter or underscore and may then contain letters, digits, or underscores.

Examples:

X
total
student1
_value
sum_1

The implemented lexical rule is conceptually:
[a-zA-Z_]([a-zA-Z0-9_])*

- ## Constant

The language recognizes:

Integer literals
10
25
100

Floating-point literals
3.14
10.50
0.25

Boolean literals
true
false

- ## Comments

The lexer supports both single-line and block comments.

- Single-line:

```
// this is a comment
```

- Block:

```
/* this is
    a block comment */
```

Comments are discarded by the lexical analyzer and are not passed to the parser.

- ## Formal Context-Free Grammar

The following CFG describes the major structure of the implemented language.

```
program
   → statement_list

statement_list
   → statement_list statement
   | statement

statement
   → declaration
   | assignment
```

```
   | print_statement
   | if_statement
   | if_else_statement
   | while_statement
   | error ;

declaration
   → type ID ;

type
   → INT
   | FLOAT
   | BOOL

assignment
   → ID = expression ;

print_statement
   → PRINT expression ;

if_statement
   → IF ( expression ) block

if_else_statement
   → IF ( expression ) block ELSE block

while_statement
   → WHILE ( expression ) block

block
   → { statement_list }

expression
   → expression OR expression
   | expression AND expression
   | NOT expression
   | expression LT expression
   | expression GT expression
   | expression LE expression
   | expression GE expression
   | expression EQ expression
   | expression NE expression
   | expression PLUS expression
   | expression MINUS expression
   | expression MULTIPLY expression
   | expression DIVIDE expression
   | expression MOD expression
   | ( expression )
   | ID
   | INT_LITERAL
   | FLOAT_LITERAL
   | TRUE
   | FALSE
```

- Sample Program

A representative program supported by ParseMaster is :

```
int x;
int y;
bool flag;
x = 10;
y = 0;
flag = true;
while (x > 0) {
    y = y + x;
    x = x - 1;
}
if (flag == true) {
    print y;
} else {
    print x;
}
```

4. Compiler Architecture

The finished compiler is organized as a pipeline of five processing stages, which together implement the six modules the manual requires.

```
           Source Code (.mc)
      |
      v
Lexical Analyzer (Flex)        -> Token Stream
      |
      v
Syntax Analyzer (Bison)        -> Abstract Syntax Tree
      |
      v
Semantic Analyzer               -> Validated AST
   + Symbol Table                  (type checking, scope checking)
      |
      v
 Intermediate Code Generator    -> Three Address Code (TAC)
```

The project source tree follows this architecture:

```
src/
├── ast/
│   ├── ast.c
│   └── ast.h
├── lexer/
│   └── lexer.l
├── parser/
│   └── parser.y
├── semantic/
```

```
│   ├── semantic.c
│   └── semantic.h
├── symbol_table/
│   ├── symbol_table.c
│   └── symbol_table.h
├── tac/
│   ├── tac.c
│   └── tac.h
└── main.c
```

The repository also contains examples, tests, a Makefile, documentation, and project configuration files.
The lexer converts source characters into tokens. Bison consumes those tokens and constructs an AST. The AST is then used by the semantic analysis phase to validate the meaning of the program. The symbol table provides information about declared identifiers and their scopes. Finally, TAC generation traverses the AST and converts valid expressions and control-flow structures into a linear intermediate representation.

5. Lexer Design

The lexical analyzer lives in src/lexer/lexer.l and is a fairly conventional Flex specification once you look past the fact that it doesn't print anything itself - every rule returns a token code back to the Bison-generated parser instead.

Token Category	Regular Expression / Rule
Identifier	[a-zA-Z_](\[a-zA-Z_]|[0-9])*
Integer constant	[0-9]+
Float constant	[0-9]+"."[0-9]+
Whitespace	[\t\r]+ (discarded)
Newline	\n (line counter incremented, discarded)
Line comment	"//".* (discarded)
Block comment	"/*"([^*]|*+[^*/])**+"/" (discarded)
Invalid token	. (reported as a lexical error with line number)

6. Parser Design

The syntax analyzer is implemented using Bison in:
src/parser/parser.y
The parser receives tokens from Flex and checks whether the sequence conforms to the grammar.

- Grammar Structure

```
The parser contains grammar rules for:

program
statement_list
statement
declaration
assignment
print_statement
if_statement
if_else_statement
while_statement
```

```
block
type
expression
Each successful grammar reduction constructs an AST node using createNode().
```

- ## Operator Precedence

Bison precedence declarations are used to resolve ambiguous expression parsing.
The implemented precedence from lower to higher priority is approximately:

Level	Operators	Associativity
1	||	Left
2	&&	Left
3	== !=	Left
4	< > <= >=	Left
5	+ -	Left
6	* / %	Left
7	!	right

The parser declares these using Bison's %left and %right directives.

Therefore:
a + b * c

is interpreted as:
a + (b * c)

rather than:
(a + b) * c

- ## Syntax Error Reporting

The parser uses:

```
void yyerror(const char *s)
{
  printf("Syntax Error at line %d: %s\n", yylineno, s);
}
```

Thus syntax errors are reported together with the current line number.

For example:
Syntax Error at line 5: syntax error

The parser also contains basic recovery:
error SEMICOLON
and uses:
yyerrok;
to continue parsing after a recoverable syntax error.

7. Abstract Syntax Tree

The Abstract Syntax Tree (AST) is an intermediate tree representation of the source program produced after successful syntax analysis. Instead of storing every grammar symbol, the AST keeps the essential structural information needed by later compiler phases such as semantic analysis and Three Address Code generation.
In ParseMaster, the AST implementation is located in:

src/ast/ast.c
src/ast/ast.h

The AST is represented using the ASTNode structure. Each node contains:

- nodeType – identifies the kind of node, such as Assignment, If, While, Identifier, or literal.
- value – stores an identifier name, literal value, or other node-specific information.
- scope – stores the scope associated with the node.
- left – points to the left child.
- right – points to the right child.
- next – links sequential statements or related nodes.

The structure allows the AST to represent both expressions and statements. The next pointer is particularly useful for representing a sequence of statements without creating unnecessary tree levels.

• AST Node Creation

The function createNode() dynamically allocates memory for a new AST node and initializes its fields. If memory allocation fails, the compiler reports an error and terminates safely.

The general process is:

1. Allocate memory for the node.
2. Store the node type.
3. Store the optional value.
4. Initialize the scope.
5. Connect the left and right child nodes.
6. Initialize the next pointer.

This allows the Bison parser to construct AST nodes while reducing grammar rules.

- AST Visualization

ParseMaster provides a recursive printAST() function for displaying the AST in a readable tree-like format. The output uses indentation and tree branches to show the relationship between parent and child nodes.

For example, an assignment such as:

x = a + b * c;

can conceptually be represented as:

```
Assignment
├── Identifier : x
└── PLUS
├── Identifier : a
└── MULTIPLY
├── Identifier : b
└── Identifier : c
```

This representation clearly demonstrates that multiplication is performed before addition according to the parser precedence rules.

Memory Management

The AST is dynamically allocated, so memory must be released after compilation. ParseMaster provides freeAST(), which recursively frees the left, right, and next nodes before releasing the current node. This prevents unnecessary memory leaks during compiler execution.

8. Symbol Table

The Symbol Table is a central data structure used to store information about identifiers declared in the source program. It is required by the semantic analysis phase to determine whether variables are declared, what their types are, and whether they are visible from a particular scope.

The implementation is located in:

src/symbol_table/symbol_table.c
src/symbol_table/symbol_table.h

Each symbol stores:

Field	Description
Name	Identifier name
Type	int,float or bool
Line	Declaration line number
Scope	Scope where the identifier was declared

Each symbol stores: The symbol table supports up to 100 symbols using the MAX_SYMBOLS constant.

- ## Symbol Table Operations

Initialization

The initSymbolTable() function initializes the symbol table by setting the symbol count to zero.

Lookup

The lookupSymbol() function searches for a symbol using both its name and scope. If a matching symbol exists, its position in the table is returned. Otherwise, -1 is returned.

Insertion

The insertSymbol() function adds a new variable to the table. Before insertion, it checks whether the same identifier already exists in the same scope. If so, the compiler reports a redeclaration error.

Printing

The printSymbolTable() function displays the stored information in a tabular form:

Name Type Line Scope

This output helps verify that declarations and scopes are being processed correctly.

- ## Scope Handling

The compiler handles variable scopes using the enter_scope() and exit_scope() functions. A new scope is created when entering a block and removed when leaving it.

When a variable is declared, insert_symbol() checks only the current scope to prevent duplicate declarations in the same block. This allows variables with the same name to be declared in different blocks. When a variable is used, lookup_symbol() first searches the current scope and then checks the outer scopes if necessary. This allows variables from outer blocks to be used inside nested blocks, while variables declared inside a nested block are not accessible after the block ends.

We tested this feature with a program containing a global variable x and a local variable y inside an if block. The output showed that x remained in the outer scope, while y was available only inside the inner block.

Before a scope is removed, exit_scope() prints its symbol table. This helps verify that variables are stored and removed correctly according to their scope.

9. Semantic Analysis

Semantic analysis verifies whether a syntactically correct program is also meaningful according to the language rules. A program can be grammatically correct but still contain semantic errors such as using an undeclared variable or assigning an incompatible value to a variable.

The semantic analyzer is implemented in:

src/semantic/semantic.c
src/semantic/semantic.h

ParseMaster performs semantic analysis by traversing the AST and using the Symbol Table to determine the meaning and type of identifiers and expressions.

The main semantic analysis function is:

semanticAnalysis(ASTNode *root, SymbolTable *table)

It starts the recursive analysis of the AST.

Rule	Detection Point	Example
Undeclared variable use	get_expr_type() on NODE_ID; also checked for assignment LHS	y = x; (x never declared)
Redeclaration	insert_symbol() returns -1 within declaration action	int x; int x;
Scope violation	lookup_symbol() only searches enclosing scopes, never sibling/child scopes	using a variable declared inside a now-closed if block
Type mismatch	check_relational(): comparing bool with a numeric type	5 < true
Invalid assignment	assignment action compares RHS type to declared symbol type	bool b; b = 5 + 3;
Invalid expression	check_arithmetic() / check_logical(): wrong operand types for the operator category	x && y where x, y are int

One intentional exception in our semantic analysis is that assigning an int expression to a float variable is allowed, as it is treated as an implicit type conversion. All other incompatible type assignments, such as assigning a bool value to a numeric variable or comparing operands of different types, are reported as semantic errors.

Each check_* function displays an appropriate error message along with the corresponding line number and increments the global semantic_errors counter. After parsing is complete, the parser checks this counter once to determine whether semantic analysis has succeeded. If no semantic errors are found, the compiler proceeds to generate the Three-Address Code (TAC).

10. Intermediate Code

After the source program has been parsed and analyzed, ParseMaster generates an intermediate representation called Three Address Code (TAC).

The TAC implementation is located in:

src/tac/tac.c
src/tac/tac.h

Three Address Code represents complex expressions and control-flow operations using simple instructions. Temporary variables are used to store intermediate results, while labels are used for conditional and loop control flow.

- ## Expressions

The generateTAC() function generates code by recursively processing an expression tree. For a binary expression, it first generates code for the left operand, then the right operand, and finally creates the instruction for the operation. The result is stored in a new temporary variable (such as t1, t2, etc.), and the temporary name is returned so it can be used in later expressions. For leaf nodes, such as identifiers and constants, no additional code is generated because they can be used directly.

```
Source:  c = a + b * 2;

 TAC:
 t1 = b * 2
 t2 = a + t1
 c = t2
```

- ## Control Flow

if, if-else, and while are all lowered using labels (L1, L2, ...) and a conditional jump of the form ifFalse <place> goto <label>, plus an unconditional goto where needed for else-branches and loop back-edges .

```
Source:
 while (x > 0) {
       y = y + x;
       x = x - 1;
 }

 TAC:
 L1:
 t1 = x > 0
 if False t1 goto L2
 t2 = y + x
 y = t2
 t3 = x - 1
 x = t3
 goto L1
 L2:
```

if-else needed one extra instruction that a plain if doesn't: a goto immediately after the then-branch, jumping past the else-branch entirely. Without it, execution would fall straight through from the end of the then-branch into the start of the else-branch, running both .

- ## Print Statements

print statements are the simplest case in the whole generator: evaluate the expression down to a place, then emit a single print <place> instruction.

11. Challenges

During the development of ParseMaster, several implementation challenges were encountered.

AST and Parser Integration

One of the main challenges was connecting the Bison grammar with AST construction. Every successful grammar reduction needed to create the appropriate AST node while maintaining the correct left, right, and next relationships.

Expression Precedence

Expressions containing multiple operators required careful precedence handling. Without precedence declarations, expressions such as:

a + b * c

could be interpreted incorrectly. Bison precedence declarations were therefore used to ensure that multiplication and division are evaluated before addition and subtraction.

Symbol Table and Scope Management

Handling variables in nested blocks required more than simply checking whether a name existed. The compiler had to associate each variable with its declaration scope and search outer scopes when resolving an identifier.

Semantic Type Checking

Supporting three data types—int, float, and bool—required different rules for arithmetic, relational, equality, and logical operations. Special handling was also required for the allowed int-to-float assignment.

TAC Control Flow

Generating TAC for if, if-else, and while statements was another challenge because high-level control structures must be transformed into labels and conditional/unconditional jumps.

Temporary and Label Management

Temporary variables and labels must remain unique during TAC generation. ParseMaster therefore maintains separate counters for temporary variables and labels and resets them before generating a new TAC sequence.

Module Integration

The final compiler integrates Flex, Bison, AST, Symbol Table, Semantic Analysis, TAC, and the main driver. The Makefile generates parser and lexer source files and then compiles all required C modules into the compiler executable.

12. Testing

Testing was performed using different input programs stored in the tests/directory. Each test file was designed to verify a specific feature of the compiler, including syntax analysis, semantic analysis, AST generation, symbol table management, and Three Address Code (TAC) generation.

Test File	Category	Expected Result
declaration.txt	Variable Declaration	Variables are declared successfully and stored in the symbol table.
assignments.txt	Assignment Statements	Valid assignments are accepted for declared variables.
arithmetic.txt	Arithmetic Expressions	Arithmetic expressions are parsed correctly and corresponding AST/TAC are generated.
relational.txt	Relational Expressions	Relational operators are parsed correctly and produce Boolean results.
logical.txt	Logical Expressions	Logical expressions with Boolean operands are accepted.
control.txt	Control Statements	if, if-else, and while statements are parsed and processed correctly.
print.txt	Print Statement	Print statements are parsed successfully.
invalid/	Invalid Programs	Syntax and semantic errors are detected and reported with line numbers.

The test cases confirmed that the compiler correctly performs lexical analysis, syntax analysis, AST construction, symbol table management, semantic analysis, and Three Address Code (TAC) generation. Invalid test programs verified that syntax and semantic errors are detected and reported with appropriate error messages and line numbers.

13. Conclusion

ParseMaster successfully demonstrates the major front-end concepts of compiler construction using Flex, Bison, and the C programming language.

The project begins with lexical analysis, where Flex converts source characters into tokens. Bison then performs syntax analysis according to the Context-Free Grammar and constructs an Abstract Syntax Tree. The AST provides a structured representation of the source program for subsequent compiler phases.

The Symbol Table stores information about identifiers, including their names, types, declaration lines, and scopes. Semantic Analysis uses this information to detect undeclared variables, scope violations, type mismatches, invalid expressions, and invalid Boolean conditions.

Finally, the Three Address Code generator converts expressions and control-flow structures into a simpler intermediate representation using temporary variables and labels. This demonstrates how a high-level program can gradually be transformed into a lower-level form suitable for later compiler stages.

The project also demonstrates the importance of modular compiler design. Each phase has its own source files and responsibilities, while the Makefile integrates the modules into a single executable compiler. The repository includes examples and tests for different language constructs and error conditions.

Although ParseMaster does not generate machine code or an executable target program, it successfully demonstrates the core front-end pipeline from source code to intermediate representation. Through this project, the team gained practical experience with lexical analysis, parsing, AST construction, symbol-table management, semantic checking, intermediate-code generation, debugging, testing, and collaborative software development.

14. References

- Compiler Construction Lab: Project Manual, Metropolitan University, Bangladesh, Department of Computer Science and Engineering.
- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). Compilers: Principles, Techniques, and Tools (2nd ed.). Addison-Wesley.
- GNU Bison Manual - https://www.gnu.org/software/bison/manual/
- Flex Manual -https://westes.github.io/flex/manual/
- Project repository -https://github.com/mashudasiddikamaisha/ParseMaster-Compiler-Construction-Lab-Project.git